SOURCE-GROUNDED TECHNICAL REFERENCE

Project Audit

Architecture, implementation evidence, validation, and unresolved risks

11 October 2026

Architecture · Execution · Debugging · Safety · Future Design

Lab revision 6527f296 · FactorySemantics pin b90ca1c8

Evidence index and documented implementation gaps

Contents

Project audit **3**
Evidence and missing requirement sources 3
Architecture and implementation inventory 3
Validation performed in this environment 4
Source-level findings and unresolved risks 4
Planned features: documented versus proposed 5
Cross-check index 6

Project audit

Evidence date: 11 October 2026 (Asia/Kolkata). Application source reviewed at lab commit `6527f29645bd2f1990cb9b8c34338e034e4c2184`; upstream pinned to `b90ca1c81e3b520be629999fff8bd9ffb82ad514`. This audit is an evidence index, not a certification of production safety or hackathon completion.

Evidence and missing requirement sources

The README names the project **AI-Driven Chaos Engineering for Industrial Resilience**. The separate backend uses the names TeamSynapse and Bosch Inference Backend. Neither naming establishes the original Bosch Hackathon problem statement.

Inspected the current tracked files, accessible workspace project files, local `work` branch and `origin/main`, all 11 reachable lab commits and their file inventories, and historical README/docs/config/backend text. Searches for problem statement, Day 2, deliverables, evaluation criteria, and recommendation engine found no authoritative requirement document. No requirement attachment was present in the accessible workspace directories. This does not establish that such documents never existed externally or in unreachable Git objects.

`PROJECT_AUDIT.md` and `EXECUTION_FLOWS.md` did not previously exist; they are created by this documentation task. `README.md`, existing design documents, definitions, and implementation supply the evidence. Original problem wording, Day 2 acceptance criteria, team ownership, deadlines, and any required Docker architecture remain **unavailable**, not inferred requirements. The user's requested future-feature topics are documentation scope, not proof of an agreed team design.

Architecture and implementation inventory

Area	Implemented evidence	Important qualification
Target MES	Pinned editable FactorySemantics package; `Lifecycle.prepare/start`; copied bottling pack; SQLite; four plant components plus log sink	Target source is unmodified; `sitecustomize` adds lab behavior inside selected processes
Experiment schema	`model.Experiment`, `Step`, `State`, `from_mapping`, `TRANSITIONS`	Syntactic validity differs from policy acceptance and actual runtime readiness
Deterministic safety	`PolicyValidator.validate`, `ResourceGate.check`, allowlist and safety-policy YAML	Policy estimate is not a strict comprehensive runtime deadline; host resource gate is sampled before runs
Controller	`Controller.run`, `_run_infrastructure`, `_run_application`, `StateTracker.to`	Application and infrastructure evidence are intentionally different
Faults	`FAULTS`: no-fault, outage, freeze, CPU pressure, DB lock, DB latency, resource limits, eight simulator faults	Three additional fault specs are explicitly unsupported; available capability depends on host
Recovery	`RecoveryVerifier.wait/evaluate_sample`; `_application_recovery`	Recovery requires application evidence; cleanup alone cannot establish recovery
Watchdog/cleanup	`watchdog.run/spawn`, `cleanup_records`, `LabApi.emergency_stop`	Same-host watchdog is not independent infrastructure; crash-window and concurrency risks below
MCP	`mcp_server.build_server`, 21 tools, stdio; `.mcp.json`	Four tools marked destructive; annotations are hints, not an approval mechanism
Agents	`Agent.run/_dispatch`, `INJECTOR`, `ANALYST`, `ChatClient`	Role tools and injection approval enforced; most procedural instructions and output schemas are prompts
Observability	`MetricsAdapter`, `Sampler`, `Collector`, log excerpts, result JSON, report/CSV, Prometheus/Grafana files	Live graphs are supplementary; run records are primary evidence

Area	Implemented evidence	Important qualification
Generic RAG backend	`app.main:app`, `RAGPipeline`, `DocumentLoader`, `FaissStore`, LangGraph	No integration from lab agents to RAG and no implemented recommendation engine
Docker	Upstream `docker/docker-compose.yml`, Dockerfile, nginx config	Lab workflow is native-process SQLite; Compose is an alternative target deployment, not lab orchestration

Validation performed in this environment

Check	Outcome and scope
Existing GitHub authentication	Read-only `gitls-remoteoriginHEAD` succeeded; no additional Git token required
Setup	`bashscripts/setup.sh` installed Python requirements and pinned MES, generated replay, initialized SQLite, passed prerequisites
Dependency consistency	Lab `.venv/bin/python-mpipcheck`: no broken requirements
Plant startup	`scripts/start.sh--with-exporter`: all four components alive, six of six machines connected
Functional HTTP	MES `/health` returned `status:ok`; exporter `/metrics` returned plant/machine/production metrics
No-fault integration	`20261010T190848Z-baseline-observe-e3835a`: `recovered`, cleanup true, authenticated read succeeded, tag IDs advanced, SQLite quick check passed
Lab suite with normal process access	**222 passed, 4 skipped** in 71.83 seconds; skipped tests require delegated user-systemd cgroup scopes
Initial sandboxed suite	**219 passed, 3 failed, 4 skipped**: process-name assertions failed under restricted `/proc` writes; passed with normal process access
Initial sandboxed plant check/baseline	Could reach API but process environment inspection raised `psutil.AccessDenied`, so correctly refused experiment preflight; normal process access passed
Unexecuted here	Real destructive outage/freeze/DB/resource fault demos; application harness integration; hosted LLM calls; RAG backend suite and real embedding/LLM calls; Compose; Prometheus/Grafana servers; upstream suite

The baseline run ID uses UTC by the application's design; it is not the user's local date. Existing documentation's historical "Verified" statements are historical claims and are not additional runs performed during this audit. Unit/component success does not prove all live faults.

Source-level findings and unresolved risks

1. **Stale architecture prose.** `docs/ARCHITECTURE.md` part D and README's "no AI engine" wording predate the implemented agents and generic RAG backend. No recommendation engine exists, but AI components do. Distinguish these claims.
2. **Missing cited artifacts.** README and target documentation refer to `docs/EXPERIMENTS.md` and `docs/RESOURCE_ASSESSMENT.md`, which are absent in the current tracked inventory. There are no historical run records committed beyond `results/.gitkeep`.
3. **Missing OPC wrapper dependency.** `scripts/inject-opc.sh` expects `injections/chaos-opcua/.venv/bin/chaos-opcua` and `configs/lab-server-stop.yaml`; that checkout is absent and main setup does not install it. The built-in `opc-server-outage-10s.yaml` is a separate available path.
4. **Lease concurrency.** `Lease.acquire` reads/checks/writes; `lock_path` exists as an attribute but is not used for mutual exclusion. Atomic replacement prevents torn JSON, not concurrent controllers both acquiring. `LabApi.submit` adds another check-before-spawn window. This is a source-visible risk, not a reproduced concurrent-run failure.
5. **Intent crash windows.** `Fault.intent` precedes injection, but some necessary undo data (CPU affinity, helper PID, resource-limit original values) is added only after side effects. Self-expiring helpers reduce duration; they do not prove full restoration after every instruction-level crash. An inherited minimal intent may be insufficient.

6. **Failure record retention.** Controller paths clear lease records after cleanup, even when cleanup outcomes fail; `finally` releases the lease. A watchdog cannot subsequently reconstruct cleared records. Emergency sweep is useful but not a full substitute for the original undo record.
7. **Watchdog timeout scope.** `watchdog.run` cleans/release records if overdue but does not terminate the controller. Long-running controller code could continue after lease release. A cleanup deadline is not a cancellation fence.
8. **Compound reporting.** Infrastructure loop stops after failed interim recovery; final verdict calculation does not explicitly require that every planned step ran. A later final recovery could obscure an incomplete compound plan. Phase summaries merge all fault windows. This needs dedicated tests before advertising reliable multi-step outcomes.
9. **Approval boundary.** `Agent._dispatch` gates non-dry `run_experiment`; direct CLI, Python API, and generic MCP calls do not use this approver. MCP annotations are metadata. "One experiment per task" and "do not retry denial" are prompt instructions, not counters or denial latches.
10. **LLM evidence/output trust.** `extract_json` returns any dictionary; no typed verdict validation. Transcript-derived emergency-stop flag and baseline-comparison check improve honesty, but most verdict fields remain model statements. Tool text is truncated at 12,000 characters. Transcripts may contain sensitive project/log content; log redaction is best effort, not a secrecy guarantee.
11. **Measurement limits.** `process_table` creates process objects and reads `cpu_percent(interval=None)` without a retained measurement interval, so component CPU percentages may be first-call zeros. API outage makes API-derived ingestion evidence unavailable. `quick_check` and nondecreasing IDs do not prove no lost business transactions. Prometheus rates smooth short windows.
12. **Configuration metadata limits.** `preconditions`, `expected_measurements`, `cleanup`, and `scoring` are recorded strings/lists, not generic executable assertions, shell hooks, or guaranteed measurement coverage. Actual built-in preconditions and category control the behavior.
13. **Backend boundaries.** `/api/v1/predict` computes input statistics and returns a fixed example detection; installed ONNX Runtime is not a loaded inference model. RAG has no authentication, broad CORS, linear local-store operations, per-worker model memory, and no distributed storage support.
14. **Host portability.** Main lab resource gate reads host memory; unlike backend memory checks it is not fully cgroup-aware. Process discovery needs environment access. Scope-dependent resource injections need user systemd. Hardcoded "Docker not installed" and "8 GB RAM" unsupported-reason text is stale on this cloud host; policy/capability refusal remains the relevant fact.

Planned features: documented versus proposed

Topic	Existing implementation	Remaining work / status
Recommendation engine	Results, compare, retrieval backend, analyst `suggested_next_experiment` field	No joined engine, structured recommendation contract, ranking or evaluation; integration designs in study guide are proposals
API-crash injection	`ProcessOutage`, API allowlist, stored 10-second SIGKILL definition, restart and staged recovery	Existing native MES capability; proposed richer evidence/UI and distinct backend/container adapters are new work
Multiple injections	`Experiment.steps`, sequential controller, compound definition, intermediate recovery	Disabled by policy; reliable completion/per-step reporting improvements needed; concurrent overlap requires new design

No original Day 2 checklist is available to classify these as satisfied team deliverables. The guide's future plan preserves the current safety constraints unless an explicitly labelled proposal would require a separately reviewed policy change.

Cross-check index

Use EXECUTION_FLOWS.md for precise paths and PROJECT_STUDY_GUIDE.md for explanations. Evidence groups: README/setup/target (§§1–4); model/policy/settings (§6); controller/faults/cleanup/recovery (§§5,7–9); MCP/agents (§§5–7); monitoring/Grafana (§9); upstream Compose and backend deployment (§10); tests and findings (§11); missing deliverables (§12); proposal boundaries (§13); interview answers (§14). Original requirements remain the unresolved external source gap.